

第三讲

Floating Point

笔记整理自 2026 年 9 月 14 日课程

1. This lecture to cover:

- Background: Fractional binary numbers
- IEEE floating point standard: Definition
- Example and properties
- Rounding, addition, multiplication
- Floating point in C
- Summary

2. Fractional binary numbers

- What is 1011.101_2 ?

$$1011.101_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} = 11.625_{10}$$

$$b_i b_{i-1} \dots b_1 b_0 . b_{-1} b_{-2} \dots b_{(-n)_2} = \sum_{i=-n}^0 b_i \times 2^i$$

- Divide by 2 by shifting right
- Multiply by 2 by shifting left
- Numbers of form $0.111111\dots_2 \rightarrow 1_{10}$

Limitations

- Can only exactly represent fractions of the form $\frac{k}{2^n}$ where k is an integer and n is a non-negative integer.
- Just one setting of binary point within w bits.

3. IEEE 754 浮点数标准

IEEE 754 规定了浮点格式、特殊值、舍入方向、基本运算的结果以及异常状态。以下主要讨论最常见的二进制格式。一个有限数的数学形式为

$$V = (-1)^s \times M \times 2^E.$$

其中 s 是符号位, M 是有效数 (significand), E 是无偏指数。位串由三个字段组成:

s	exp	frac
符号位	有偏指数域	小数域 (fraction field)

注意: frac 只保存有效数的小数部分; 它并不总等于 M 。规格化数有隐含的首位 1, 非规格化数有隐含的首位 0。

3.1. 常见二进制格式

格式	总位数	s	exp	frac	精度 p	偏置
binary16	16	1	5	10	11	15
binary32 (C 的常见 float)	32	1	8	23	24	127
binary64 (C 的常见 double)	64	1	11	52	53	1023
binary128	128	1	15	112	113	16383

这里 p 包含规格化数不存储的隐含首位。binary32 约有 7 位可靠十进制有效数字，binary64 约有 15–16 位；这描述的是精度，不是数值范围。

例：binary32 的 $p = 24$ 。在区间 $[1, 2)$ 中，相邻可表示数相差 2^{-23} ；因此 1 的下一个较大浮点数是 $1 + 2^{-23}$ 。但在 $[2, 4)$ 中相邻数相差 2^{-22} 。指数增大一，相邻数的绝对间距也扩大一倍。

3.2. 按指数域分类与解释规则

设指数域有 k 位、小数域有 n 位，

$$\text{Bias} = 2^{k-1} - 1, \quad F = \sum_{i=1}^n f_i 2^{-i}.$$

所有位模式都由下面的规则唯一分类：

exp	frac	类别	数值或含义
0	0	零	$(-1)^s \times 0$
0	$\neq 0$	次正规数	$(-1)^s \times F \times 2^{1-\text{Bias}}$
$1 \dots 2^k - 2$	任意	规格化数	$(-1)^s \times (1 + F) \times 2^{\text{Exp}-\text{Bias}}$
$2^k - 1$	0	无穷	$(-1)^s \infty$
$2^k - 1$	$\neq 0$	NaN	不是一个实数值

3.2.1. 规格化数 (normal number)

规则：当 $1 \leq \text{Exp} \leq 2^k - 2$ 时，

$$E = \text{Exp} - \text{Bias}, \quad M = 1 + F = 1.f_1 f_2 \dots f_{n\{2\}}.$$

隐含首位 1 不占存储位，所以 binary32 虽只有 23 位 frac，却有 24 位二进制精度。

例 1：编码十进制 15213 为 binary32。

$$15213_{10} = 11101101101101_2 = 1.1101101101101_2 \times 2^{13}.$$

因此 $s = 0$, $E = 13$, $\text{Exp} = 13 + 127 = 140 = 10001100_2$ ，并在小数域末尾补零：

$$0 \ 10001100 \ 110110110110100000000000$$

它是精确编码，因为整数的二进制有效位数没有超过 24。

例 2：解码 binary32 位模式 0xC0A00000。

其位串为 1 10000001 010000000000000000000000。于是

$$s = 1, \quad E = 129 - 127 = 2, \quad M = 1.01_2 = 1 + 2^{-2} = 1.25,$$

$$V = (-1)^1 \times 1.25 \times 2^2 = -5.$$

3.2.2. 次正规数 (subnormal number)

旧资料也常称 denormalized number。规则：当 $\text{Exp} = 0$ 且 $\text{frac} \neq 0$ 时，

$$E = 1 - \text{Bias}, \quad M = F = 0.f_1f_2\dots f_{n\{2\}}.$$

指数不是 $0 - \text{Bias}$ ，有效数也没有隐含首位 1。这样，最大次正规数与最小规格化数之间仍保持同一间距，实现渐进下溢 (gradual underflow)。

例 1：binary32 的最小正次正规数的 frac 只有最低位为 1，因此

$$V = 2^{-23} \times 2^{-126} = 2^{-149} \approx 1.4013 \times 10^{-45}.$$

例 2：最大正次正规数为

$$(1 - 2^{-23}) \times 2^{-126} = 2^{-126} - 2^{-149}.$$

最小正规格化数为 1×2^{-126} 。二者之差正好是 2^{-149} ，与该区域相邻次正规数的间距一致。

例 3：解码 0x001C0000。位串分段为

0 00000000 0011100000000000000000

$F = 2^{-3} + 2^{-4} + 2^{-5} = \frac{7}{32}$ ， $E = -126$ ，所以

$$V = \left(\frac{7}{32}\right) \times 2^{-126} = 7 \times 2^{-131} \approx 2.5714 \times 10^{-39}.$$

3.2.3. 有符号零

规则： $\text{Exp} = 0$ 且 $\text{frac} = 0$ 表示零； $s = 0$ 得 $+0$ ， $s = 1$ 得 -0 。数值比较时 $+0 = -0$ ，但其位模式和部分运算结果不同。

例：

- $\frac{1}{+0} = +\infty$ ，而 $\frac{1}{-0} = -\infty$ ；
- `copysign(1.0, -0.0)` 得到 -1 ；
- 通常的相等比较中， $+0.0 == -0.0$ 为真。

有符号零能保存趋近方向或符号信息。例如一个极小负数在向零舍入后可成为 -0 。

3.2.4. 无穷大

规则：指数域全 1 且小数域全 0。符号位分别给出 $+\infty$ 和 $-\infty$ 。

例：

- 在默认舍入模式下，最大有限正数再乘 2 会发生上溢，通常得到 $+\infty$ ；
- $\frac{1}{+0} = +\infty$ ， $-\frac{1}{+0} = -\infty$ ；
- $\infty + 3 = \infty$ ，但 $\infty - \infty$ 没有确定的数值结果，得到 NaN。

3.2.5. NaN

规则：指数域全 1 且小数域非零。NaN 分为 quiet NaN (qNaN) 和 signaling NaN (sNaN)；具体如何用 frac 的最高位区分由格式约定。NaN 的其余小数位可作为 payload 保存诊断信息，但跨运算、语言或平台时不应假定 payload 必然保留。

例： $\frac{0}{0}$ 、 $\sqrt{-1}$ (在实数浮点运算中)、 $\infty - \infty$ 、 $0 \times \infty$ 都产生 NaN，并引发 invalid 异常。qNaN 通常继续传播而不重复触发 invalid；使用 sNaN 通常触发 invalid，并把结果静默化为 qNaN。

NaN 的比较规则很重要：除“不等于”外，普通有序比较都返回假。因此若 x 为 NaN，则 $x == x$ 也为假， $x < 0$ 与 $x \geq 0$ 同样都为假。检查 NaN 应使用语言提供的 `isnan`，而不是依赖大小比较。

3.3. 重要边界值

数值	binary32	binary64
最小正次正规数	2^{-149}	2^{-1074}
最小正规格化数	2^{-126}	2^{-1022}
最大有限数	$(2 - 2^{-23})2^{127}$	$(2 - 2^{-52})2^{1023}$
1 的下一个较大数	$1 + 2^{-23}$	$1 + 2^{-52}$

machine epsilon 一词常指使 $1 + \varepsilon > 1$ 的最小间距，因此 binary32 常取 $\varepsilon = 2^{-23}$ ，binary64 常取 $\varepsilon = 2^{-52}$ 。它不是整个数轴上的统一绝对误差上界。

3.4. 舍入规则

IEEE 754 的运算语义可理解为：先求无限精度的精确结果，再按目标格式与当前舍入方向得到可表示结果。2019 版标准定义五种舍入方向；常见硬件至少直接支持前四种。

3.4.1. 舍入到最近，正中取偶 (roundTiesToEven)

选择距离精确值最近的可表示值；若恰好位于两个可表示值的中点，则选择最低保留位为 0 的那个。它是二进制运算的默认舍入模式。

十进制演示（保留到整数）： $1.4 \rightarrow 1$, $1.6 \rightarrow 2$, $1.5 \rightarrow 2$, $2.5 \rightarrow 2$, $-1.5 \rightarrow -2$ 。最后三个中点例子中，结果均选择偶数。

二进制例子（保留小数点后两位）：

- 10.00011_2 距 10.00_2 更近，结果为 10.00_2 ；
- 10.00110_2 距 10.01_2 更近，结果为 10.01_2 ；
- 10.11100_2 恰在 10.11_2 与 11.00_2 中间，后者最低位为 0，结果为 11.00_2 ；
- 10.10100_2 恰在 10.10_2 与 10.11_2 中间，前者最低位为 0，结果为 10.10_2 。

3.4.2. 舍入到最近，正中远离零 (roundTiesToAway)

选择最近值；正中点选择绝对值较大的结果。这一方向是 IEEE 754-2008/2019 规定的属性之一，但并非所有二进制硬件环境都提供动态切换支持。

例（保留到整数）： $1.5 \rightarrow 2$, $2.5 \rightarrow 3$, $-1.5 \rightarrow -2$, $-2.5 \rightarrow -3$ 。非中点如 1.4 仍舍入为 1。

3.4.3. 向零舍入 (roundTowardZero)

选择绝对值不大于精确结果的、最接近零的可表示数；等价于丢弃超出精度的部分。

例（保留到整数）： $1.9 \rightarrow 1$, $-1.9 \rightarrow -1$ 。发生正溢出时，该方向返回最大有限正数而不是 $+\infty$ ；负溢出相应返回最小有限负数。

3.4.4. 向正无穷舍入 (roundTowardPositive)

选择不小于精确结果的最小可表示数。

例（保留到整数）： $1.1 \rightarrow 2$, $-1.9 \rightarrow -1$ 。注意它并非“增大绝对值”：负数结果会趋近于零。

3.4.5. 向负无穷舍入 (roundTowardNegative)

选择不大于精确结果的最大可表示数。

例（保留到整数）： $1.9 \rightarrow 1$, $-1.1 \rightarrow -2$ 。负数结果的绝对值可能增大。

3.4.6. Guard、Round 与 Sticky 位

硬件通常保留额外位来判断舍入。设最终保留位之后的第一位为 round bit, 之后所有位的逻辑或为 sticky bit; 一些教材把最终保留位称为 guard bit。对于正中取偶:

- round bit 为 0: 被删除部分小于半个单位, 不增加保留部分;
- round bit 为 1 且 sticky 为 1: 大于半个单位, 保留部分加 1;
- round bit 为 1 且 sticky 为 0: 恰好为中点, 仅当最低保留位为 1 时加 1, 使结果最低位变为 0。

例: 把 1.0011000_2 保留为 4 位有效数字 ($1.xxx$)。保留部分是 1.001_2 , round bit 为 1, 其后还有 1, 所以结果加一成为 1.010_2 。把 1.1111100_2 同样舍入时得到 10.000_2 , 此时还要右移并把指数加一, 这一步称为舍入后的再规格化。

3.5. 五类浮点异常

IEEE 754 的异常通常设置状态标志, 并交付标准规定的默认结果; 是否陷入 (trap) 由运行环境决定。一个操作可能同时设置多个标志。

3.5.1. Invalid operation (无效运算)

精确结果没有可定义的浮点数值, 默认结果通常是 qNaN。

例: $\frac{0}{0}$ 、 $\frac{\infty}{\infty}$ 、 $\infty - \infty$ 、 $0 \times \infty$ 、负数的实数平方根, 以及把 sNaN 用作操作数。这里不是因为结果过大或过小, 而是运算本身没有确定的实数或扩展实数结果。

3.5.2. Division by zero (除以零)

有限且非零的被除数除以精确的零时触发, 默认结果是带正确符号的无穷。

例: $\frac{3}{+0} = +\infty$, $\frac{3}{-0} = -\infty$, 并设置 divideByZero 标志。 $\frac{0}{0}$ 不属于此类, 而属于 invalid。

3.5.3. Overflow (上溢)

舍入前的有限结果在目标格式中指数过大。默认正中取偶时, 结果通常为相应符号的无穷, 并同时设置 inexact; 定向舍入可能返回最大有限数。

例: binary32 最大有限数 $(2 - 2^{-23})2^{127}$ 乘 2。正中取偶得到 $+\infty$; 向零舍入得到最大有限 binary32。

3.5.4. Underflow (下溢)

有限结果非常小且发生精度损失。标准把 underflow 与 tininess、inexact 联系起来; 实现可在舍入前或舍入后检测 tininess, 因此只得到一个精确的次正规结果不一定设置 underflow。

例: binary32 的 $\frac{2^{-149}}{2} = 2^{-150}$ 位于 0 与最小正次正规数 2^{-149} 的正中间。正中取偶得到 +0 (零的最低位为偶), 并设置 inexact 与 underflow。

3.5.5. Inexact (不精确)

舍入后的结果不等于无限精度的精确结果。这是最常见的异常标志。

例: binary32 中计算 $\frac{1}{10}$ 。因为 0.1 的二进制展开无限循环, 只能保存其邻近值, 所以设置 inexact。若两个可表示数相加的精确结果仍可表示, 例如 $1 + 0.5 = 1.5$, 则不设置 inexact。

3.6. 基本运算规则

3.6.1. 加法与减法

设 $x = (-1)^{s_1} M_1 2^{E_1}$, $y = (-1)^{s_2} M_2 2^{E_2}$, 并假设 $E_1 \geq E_2$:

- 1) 将较小指数的有效数右移 $E_1 - E_2$ 位, 使二进制点对齐;
- 2) 按符号对有效数加减;
- 3) 将结果规格化;

- 4) 按目标精度舍入；若舍入产生进位，再次规格化；
- 5) 根据结果设置 overflow、underflow 或 inexact 等标志。

例（只有 4 位有效数字）：

$$\begin{aligned} & 1.010_2 \times 2^2 + 1.110_2 \times 2^3 \\ &= 0.1010_2 \times 2^3 + 1.1100_2 \times 2^3 \\ &= 10.0110_2 \times 2^3 = 1.00110_2 \times 2^4. \end{aligned}$$

保留 4 位有效数字并正中取偶得到 $1.010_2 \times 2^4$ 。

浮点加法满足交换律（忽略 NaN payload 等表示细节），但不满足结合律。

反例：在精度不足以保留 $10^{10} + 3.14$ 的低位时， $(3.14 + 10^{10}) - 10^{10}$ 可得到 0，而 $3.14 + (10^{10} - 10^{10}) = 3.14$ 。

3.6.2. 乘法

有限非零数相乘时，精确中间结果满足

$$s = s_1 \oplus s_2, \quad M = M_1 M_2, \quad E = E_1 + E_2.$$

然后规格化、舍入并检查指数范围。

例（4 位有效数字）：

$$(1.010_2 \times 2^2)(1.110_2 \times 2^3) = 10.0011_2 \times 2^5 = 1.00011_2 \times 2^6.$$

保留 4 位有效数字，删除部分为 11，故舍入为 $1.001_2 \times 2^6$ 。

乘法不满足结合律，也不对加法满足分配律。

结合律反例：在有限格式中， $(10^{20} \times 10^{20}) \times 10^{-20}$ 可能先上溢为 ∞ ；而 $10^{20} \times (10^{20} \times 10^{-20})$ 可得到约 10^{20} 。

分配律反例： $10^{20} \times (10^{20} - 10^{20}) = 0$ ，但 $(10^{20} \times 10^{20}) - (10^{20} \times 10^{20})$ 可能成为 $\infty - \infty = \text{NaN}$ 。

3.6.3. 融合乘加 (FMA)

$\text{fma}(a, b, c)$ 计算 $a \times b + c$ ，只在最终结果处舍入一次；普通写法 $a*b+c$ 允许乘法结果先舍入，再进行加法。

例：若 $a \times b$ 的精确值含有目标格式无法保存的低位，而这些低位会与 c 发生抵消，则 FMA 能在相加前保留它们，可能得到比两次舍入更准确、甚至不同的结果。因此编译器把 $a*b+c$ 改成 FMA 会改变逐位结果，严格浮点语义下需要相应许可。

3.7. 比较、次序与代数性质

- 除 NaN 外，有限数和无穷可按数值有序比较； -0 与 $+0$ 数值相等。
- 任一 NaN 参与 $<$ 、 $<=$ 、 $>$ 、 $>=$ 、 $==$ 时结果均为假； $!=$ 为真。
- IEEE 754 还定义 `totalOrder`，可以对 -0 、 $+0$ 、无穷和不同 NaN 表示建立全序；它不同于普通语言比较运算符。
- 加法和乘法通常是交换的，但不结合；乘法不对加法分配。
- 有限浮点运算不对实数运算封闭，因为结果可能为无穷或 NaN。

例：若 x 为 NaN，条件 $x < 1 \ || \ x >= 1$ 为假，而实数中对应命题恒真。这是因为 NaN 是 unordered，而不是“小于”或“大于”任何数。

3.8. 十进制数为何经常不能精确表示

有限二进制小数恰好是 $\frac{m}{2^q}$ 形式。最简分数的分母若含有 2 以外的质因子，其二进制展开就无限循环。

例：

$$0.1_{10} = \frac{1}{10} = 0.0001100110011... \{ \}_2.$$

binary64 中通常保存的是邻近值

0.10000000000000000555....

因此连续三次相加 $0.1 + 0.1 + 0.1$ 不保证逐位等于由十进制 0.3 转换得到的浮点值。判断近似计算结果时应根据问题尺度选择绝对误差或相对误差容限，不能统一使用一个固定的小数常量。

4. C 中的浮点数

4.1. 类型与表达式

C 语言至少提供 `float`、`double` 和 `long double`，但语言标准本身不要求它们必须分别等同于 IEEE binary32、binary64 与某一固定扩展格式；应通过实现文档、`sizeof`、`FLT_RADIX`、`FLT_MANT_DIG`、`DBL_MANT_DIG` 等确认。

没有后缀的浮点常量如 0.1 通常具有 `double` 类型，`0.1f` 才是 `float`。许多表达式中 `float` 会经通常算术转换参与运算；可观察的求值精度还受实现、编译选项和是否使用 FMA 影响。

4.2. 类型转换规则与例子

4.2.1. 整数转浮点

若值可精确表示，则结果精确；否则舍入为相邻可表示值。binary32 具有 24 位精度，所以所有绝对值不超过 2^{24} 的整数都能精确表示，但更大的整数不一定能。

例：16777216 (2^{24}) 能精确转为 binary32；16777217 夹在相邻可表示值之间，正中取偶时转为 16777216.0f；16777218 可以精确表示。

典型 32 位 `int` 的所有值都可精确转为 binary64，因为 binary64 有 53 位精度。

4.2.2. 浮点转整数

C 的转换丢弃小数部分，即向零截断。若有限值截断后不能在目标整数类型中表示，或源值是 NaN/无穷，则行为未定义；不能依赖某台机器恰好返回的值。

例：(int)3.9 得 3，(int)-3.9 得 -3。但 (int)1e100 或 (int)NAN 没有可移植结果。

4.2.3. 不同浮点格式之间

从较窄格式转到能精确包含其全部值的较宽格式通常精确；从较宽格式转到较窄格式可能舍入、上溢或下溢。

例：binary32 值先转 binary64 再转回 binary32，数值不变 (NaN payload 等表示细节除外)。binary64 的 $1.0 + 2^{-30}$ 转为 binary32 时会舍入为 1.0f，因为该增量远小于 binary32 在 1 附近的间距 2^{-23} 。

4.3. 位模式解码的可靠做法

在 C 中不要用违反严格别名规则的指针强制转换读取浮点位模式。可使用 `memcpy` (C20 以前的通用做法) 或实现提供的位转换工具。

```
#include <stdint.h>
#include <string.h>

float x = -5.0f;
uint32_t bits;
memcpy(&bits, &x, sizeof bits);
/* 在采用 IEEE binary32 的实现上, bits 为 0xC0A00000. */
```

手工解码步骤是：取最高位为 s ，接下来的指数位解释为无符号整数 Exp ，其余为 frac ；再按“按指数域分类与解释规则”中的表选择公式。必须先分类，不能对零、次正规数、无穷或 NaN 一律套用规格化公式。

4.4. 常见判断题

设 `int x; float f; double d;`，并假设 f, d 不是 NaN；在常见的 32 位 `int`、`binary32 float`、`binary64 double` 环境中：

- `x == (int)(float)x`：不恒真。例： $x = 16777217$ ，转 `float` 后成为 16777216。
- `x == (int)(double)x`：恒真，因为 `binary64` 能精确表示所有 32 位整数。
- `f == (float)(double)f`：对所有非 NaN `float` 恒真，扩大再缩回不会丢失原值。
- `d == (double)(float)d`：不恒真，第一次转换可能丢失 `binary64` 的低位。
- `f == -(-f)`：对非 NaN 值恒真；两个负号也正确恢复有符号零。
- `2/3 == 2/3.0`：通常为假。左侧是整数除法，结果为 0；右侧为浮点除法，约为 0.6667。
- `d*d >= 0.0`：在 d 不是 NaN 的前提下恒真。有限 d 的平方为非负有限数、正零或上溢后的正无穷；无穷的平方仍为正无穷。
- `(d+f)-d == f`：不恒真。若 $|d|$ 很大，加入较小的 f 时 f 会在第一次舍入中丢失。

4.5. 总结

- 位模式必须先按 `exp` 与 `frac` 分类，再使用对应公式；规格化数和次正规数的隐含首位、指数规则不同。
- IEEE 754 明确定义特殊值、舍入方向、默认结果与异常标志，使浮点行为可以独立于具体硬件进行推理。
- 浮点运算相当于精确运算后舍入，但有限精度会破坏结合律与分配律。
- 数值范围、有效精度和相邻数间距是不同概念；误差分析必须考虑数值所在的指数区间。
- C 的类型不应未经确认就等同于某个 IEEE 格式，转换越界、NaN 比较和近似相等判断尤其需要显式处理。